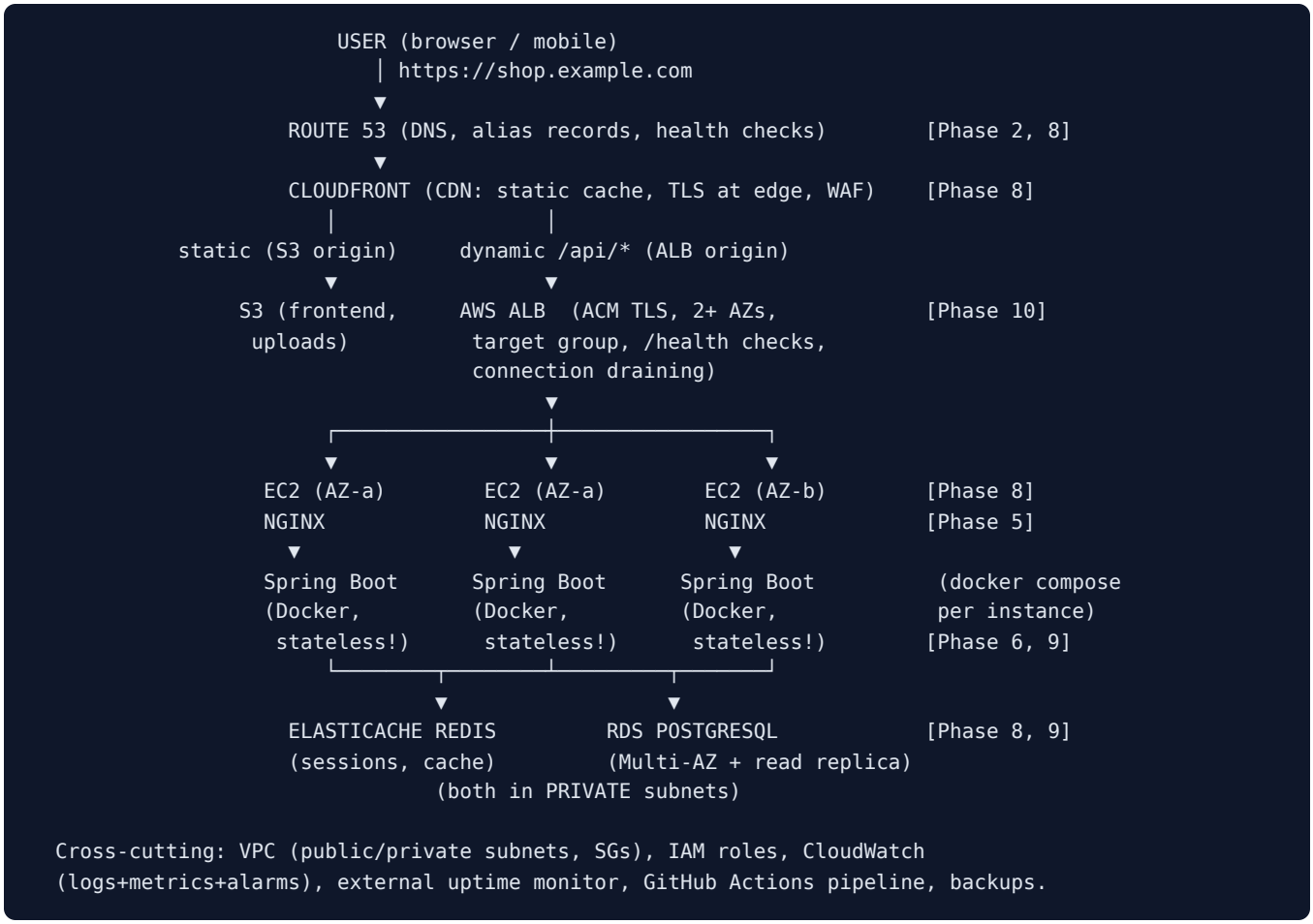
Phase 12 — Final Production Project (Capstone)

> *Build, deploy, secure, monitor, and operate the full stack. Everything from Phases 0–11, integrated.*

---

1. Target architecture



The app: a small but real **Spring Boot e-commerce API** (products, carts, orders, auth) + a static frontend (any simple SPA or even plain HTML) — the domain doesn't matter; the operations do.

2. Build order (milestones — check off in README)

M1 — The application (local) [Phases 0,3,6]

- Spring Boot API: JWT or Redis-session auth, products CRUD, orders; `/actuator/health` with liveness/readiness groups; proper status codes (Phase 3!).
- Stateless by construction: Spring Session→Redis (or JWT), uploads→S3 API (localstack or real), no local files, no in-memory state.
- Dockerfile (multi-stage) + docker compose (nginx, api, postgres, redis) — all green locally.
- **Gate:** `docker compose up -d` on a clean machine serves the API through NGINX.

M2 — Network foundation (AWS) [Phase 8]

- VPC 10.0.0.0/16: 2 public + 2 private subnets across 2 AZs; IGW; route tables. (NAT Gateway optional — note the cost; alternative: instances in public subnets with strict SGs for the learning budget.)
- SG chain: `alb-sg(80,443←world)` → `app-sg(80←alb-sg, 22←your IP)` → `db-sg(5432←app-sg)`, `redis-sg(6379←app-sg)`.
- **Gate:** diagram committed; SG rules reviewed — could you explain every line to an auditor?

M3 — Data tier [Phase 8,9]

- RDS PostgreSQL (Multi-AZ if budget allows; else document the difference), private subnets; ElastiCache Redis.
- **Gate:** unreachable from your laptop; reachable from an app instance (`nc -zv` both ways — prove it).

M4 — App tier [Phase 4,5,6,7]

- 2× EC2 across AZs; user-data bootstraps: docker, compose file (nginx+api only now — db/redis are managed), CloudWatch agent. Instance IAM role: S3 + ECR + CloudWatch, **no keys**.
- Push image to ECR; instances pull by tag.
- **Gate:** `curl <instance-private-ip>/actuator/health` = UP from within the VPC, on both.

M5 — Traffic tier [Phase 10,2]

- ALB: ACM cert, 443 listener (+80→301), target group on /actuator/health, draining 30 s.
- Route 53: `api.shop.example.com` → alias → ALB.
- **Gate:** kill one instance during an `ab` run → zero failed requests, TG shows self-recovery.

M6 — Edge + frontend [Phase 8]

- S3 (private) + CloudFront OAC for the frontend; CloudFront behavior `/api/*` → ALB origin (no caching or short TTL + auth header forwarding).
- **Gate:** one domain serves both: `shop.example.com` (cached static, fast worldwide) and `shop.example.com/api/...` (dynamic).

M7 — Observability [Phase 8,7]

- CloudWatch: app+nginx logs shipped; dashboard (ALB RequestCount, TargetResponseTime p95, 5xx, RDS CPU/connections, Redis memory); alarms→SNS→email: ALB-5xx, unhealthy-hosts, RDS-storage, billing.
- External uptime check on /health.
- **Gate:** stop one app container by hand → you receive an alert *before* checking the console.

M8 — Pipeline [Phase 7,10]

- GitHub Actions: on push to main → test → build image → push ECR (tag = git sha) → rolling deploy (SSM RunCommand or a deploy.sh over SSM/SSH: instance-by-instance, drain via TG deregister, update, health-verify, re-register).
- Rollback = redeploy previous sha (document the command!).
- **Gate:** a code change reaches production with zero downtime and no manual SSH; a deliberate bad build (failing health check) auto-stops the rollout.

M9 — Security & backup hardening [Phase 7,8]

- Checklist: no SSH passwords, SSM Session Manager preferred over open 22; secrets in SSM Parameter Store/Secrets Manager (not .env in git!); S3 buckets private + OAC; RDS automated backups + manual snapshot tested via restore; `aws s3 cp` nightly pg_dump as second layer; security headers + rate limiting in NGINX (Phase 5 block).

- **Gate:** the restore drill — recreate the DB from a snapshot into a fresh instance and point a test app at it. Timed. Documented.

M10 — The final exam: chaos day

Run these and write an incident report for each (symptom → diagnosis commands → resolution → prevention):

1. `docker stop api` on one instance (does anyone notice?)
2. Terminate an entire EC2 instance mid-traffic.
3. RDS reboot-with-failover (Multi-AZ) — measure app impact.
4. Fill a disk with logs on one instance.
5. Deploy a build whose readiness check fails.
6. Revoke the wrong SG rule and watch a tier go dark — then find it via monitoring only.

3. Deliverables (your portfolio)

1. **Repo**: app + Dockerfiles + compose + nginx configs + .github/workflows + /docs.
2. **Architecture doc**: the diagram + one paragraph per component: *why it exists, what breaks without it* — in your own words. This document is interview gold.
3. **Runbooks**: deploy, rollback, restore-from-backup, "site is down" decision tree (the Phase-1 ladder: DNS→CDN→LB→NGINX→app→DB, with the exact command at each hop).
4. **Incident reports** from chaos day.
5. **Cost sheet**: monthly cost of the full stack; a minimized lab variant; the Hetzner-equivalent — and one page arguing when each is right (you'll sound like a senior in interviews).

4. Success criteria — you are done when you can…

- Narrate a request from typed URL to rendered response naming **every** hop, protocol, and port — without notes.
- Deploy with one git push; roll back in under 2 minutes.
- Lose any single instance, AZ-component, or container with zero user impact — and *prove it with the load test running*.
- Restore the database from backup, timed, documented.
- Answer: "why NGINX if there's an ALB?", "why is Redis required for scaling here?", "where exactly does TLS terminate and why?", "what are your RPO and RTO?"

5. What's next after the course

- Infrastructure as Code: **Terraform** the whole M2–M6 build (the natural sequel — clicking consoles doesn't scale).
- The same app on **EKS** (Phase 11 for real) with GitOps (ArgoCD).
- Deeper observability: Prometheus + Grafana, tracing (OpenTelemetry).
- Then: your AI App Engineer roadmap — this infrastructure knowledge is exactly what serving AI applications needs (GPU instances, queues, streaming responses — same principles).

Course complete. Ship something real, break it on purpose, fix it calmly — that's the whole job. 🚀